Transformer-Driven Pangenomic Framework for Genomic Variant Pathogenicity (Alzheimer's genes).

HIGH LEVEL DESIGN

24AI798: Case Study with Seminar

Submitted by

Register No	Name of Student
AM.SC.P2ARI25020	Harikrishnan S

Under the Guidance of

Prathibhamol C.P

School of Computing, Amrita Vishwa Vidyapeetham



Signature of the Guide

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AMRITA VISHWA VIDYAPEETHAM

Amritapuri Campus (INDIA)

October 2026

Contents

System Overview and Objective 5

System Modules and Architecture 7

Datasets 12

Technology Stack 13

List of Figures

Figure 2.1: High-Level Block Diagram of the Transformer-Driven Pangenomic Framework 10

Figure 2.2: System Flowchart of the Genomic Variant and Gene Sequence Classification Process 11

List of Tables

Table 4.1: Technology Stack for the Transformer-Driven Pangenomic Framework 13

Chapter 1

System Overview and Objective

This project develops a transformer-driven genomic sequence classification framework with an application focus on Alzheimer's disease-related genes, particularly APP, PSEN1 and PSEN2. The engineering task is to convert genomic representations into model-ready inputs, learn a binary classification function and evaluate predictions using held-out labelled data. Genomic variants can occur in different sequence contexts, and a pangenome graph can represent alternative sequence paths rather than relying on one linear reference alone. The project investigates this representation through a graph-to-sequence pipeline and provides a separate classical machine-learning workflow for gene-sequence experiments.

The transformer workflow reads a GFA/rGFA pangenome graph, constructs a directed graph and contracts strongly connected components to obtain a directed acyclic graph. Source-to-sink traversal and seeded random walks generate candidate paths, which are linearized into DNA sequences. Overlapping k-mers are mapped to the selected DNABERT checkpoint's vocabulary, encoded with special tokens and attention masks, and supplied to a BERT-based binary sequence classifier. When verified labels are available, the model can be fine-tuned and evaluated; otherwise, a trained checkpoint supports prediction without producing unsupported accuracy scores.

The second workflow loads FASTA sequences organized by gene and class and learns classical classifiers from engineered sequence features. It combines k-mer frequencies, nucleotide composition, reference-relative mutation/context descriptors and gene indicators. A Flask dashboard supports dataset auditing, training, model comparison and prediction for uploaded FASTA records. The corresponding command-line interface exposes additional experimental settings. A and B are two execution paths within one project.

The design separates data preparation, representation, training, inference and evaluation so that each stage can be inspected and tested. The transformer does not directly consume graph adjacency; it classifies sequences extracted from the graph. The current high-level design documents implemented software behavior and the intended evaluation process. It does not assume that graph-derived inputs outperform sequence-only models or that either workflow establishes a clinical diagnosis.

Main objectives

1. Construct a modular pipeline from pangenome graph records to DNA sequence representations suitable for transformer input.

2. Use a checkpoint-compatible DNABERT tokenizer and binary classifier for labelled pathogenicity experiments.

3. Provide a classical baseline workflow and a browser interface for APP, PSEN1 and PSEN2 sequence classification.

Main objectives (continued)

4. Compare classical models and feature families using the same training-only validation folds.

5. Keep model fitting, model/threshold selection and held-out test evaluation separate.

6. Produce predictions, diagnostic visualizations and reproducible experiment artifacts without inventing performance results.

7. Identify annotation, sequence-window and biological independence requirements that must be resolved before scientific generalization claims.

Design scope

The project has two input contracts. The graph workflow accepts a pragmatic subset of GFA/rGFA and optional indexed annotations in CSV, TSV or VCF format. The classical workflow expects FASTA files under APP, PSEN1 and PSEN2 class directories. Both use integer classes 0 and 1, but the transformer labels BENIGN/PATHOGENIC and the folder-derived labels Healthy/Unhealthy have different provenance. Their meanings are not automatically interchangeable, and a common biological cohort would be needed for a controlled comparison.

The implementation is a local research prototype. The command-line graph pipeline and the Flask application are launched separately; the web application does not currently call DNABERT. Graph and sequence intermediates are kept in memory. DNABERT training saves a checkpoint, tokenizer and experiment record, while the active web pipeline is retained only in the Flask process. A database, distributed training service, background job queue and model registry are not implemented.

Evaluation approach

DNABERT experiments group identical encoded inputs before stratified train/validation/test separation. Validation selects the prediction threshold, and final metrics are computed on the held-out test partition. Classical experiments reserve a gene-and-class-stratified test set, compare candidates with five-fold validation on training data, refit the winner and then evaluate on the test set. Learned references and preprocessing steps are fitted inside the folds.

The design emphasizes experiment integrity rather than an assumed accuracy target. Exact-input checks do not establish patient-level, family-level or source-level independence. Labels, sequence alignment and data provenance still require verification. Performance conclusions must come from completed, reproducible experiments on appropriate cohorts; no measured accuracy is claimed by this high-level design report.

Chapter 2

System Modules and Architecture

The project has six stages: input, preprocessing, representation, training/selection, inference/prediction and evaluation/output. Each has graph-transformer and FASTA-classical execution paths. Figure 2.1 shows the architecture; Figure 2.2 shows execution flow.

1. Data acquisition and annotation module

The graph path reads GFA/rGFA files and indexed annotations through annotation_loader.py. CSV/TSV sequence IDs and VCF record indices require verified correspondence to extracted paths. The optional download_dataset.py helper creates demonstration allele-bubble graphs with synthetic flanks. The classical loader reads APP, PSEN1 and PSEN2 FASTA files and derives labels from class subfolders.

Methods: Read inputs, normalize annotations and collect metadata through CLI/web settings.

Data structures: GFAGraph records, sequence strings, integer label maps, gene names, filenames and PipelineConfig.

2. Graph construction and path extraction module

graph_processing.py parses H, S, L and P records into a NetworkX directed graph. SCC contraction joins member sequences in sorted order to produce a DAG. Capped source-to-sink DFS and seeded random walks generate deduplicated paths; sequence_processing.py concatenates their node DNA. Named paths are parsed but not used by the extraction controller.

Methods: GFA parsing, directed graph construction, SCC contraction, bounded traversal, random-walk sampling and path linearization.

Data structures: NetworkX DiGraph, SCC membership map, ordered node-path lists and DNA sequence lists.

3. Tokenization and transformer-input encoding module

KmerTokenizer generates stride-one overlapping k-mers using checkpoint-aligned IDs. Encoding checks for usable tokens, adds classification/separator tokens and pads or truncates to length 128 by default. Unknown k-mers use the unknown token. Truncation produces a warning because only the retained prefix reaches the model.

Methods: Tokenize, validate vocabulary, add special tokens and create padded IDs/masks.

Data structures: Token strings, vocabulary dictionaries, EncodedSequence objects and input/mask tensors of shape [batch, length].

4. Classical feature preparation module

The classical workflow uses GeneFeatures in validation_pipeline.py to construct fold-fitted sequence features. For each gene, the first healthy sequence in the training fold becomes the local reference. Feature families include global normalized k-mer frequencies, six composition statistics, eleven mutation descriptors, local-context k-mers and three gene indicators. At k = 4, the complete representation has 532 features. Reference-relative differences are position-based and assume compatible sequence coordinates; the module is not a sequence-alignment or clinical variant-calling system.

Methods: Fit healthy references on training folds, compute k-mer/composition/context descriptors, optionally select features and fit StandardScaler within the pipeline.

Data structures: Sequence/gene record arrays, reference dictionaries, dense feature matrices, fitted selectors and scalers.

5. Model training and selection module

DNABERT fine-tuning uses BertForSequenceClassification with two outputs, class-weighted cross-entropy and AdamW. Defaults are 10 epochs, learning rate 0.00002 and batch size 8, with CPU or available CUDA execution. Identical encoded inputs are grouped before stratified splitting. The test fraction defaults to 0.30 and validation receives 0.20 of the remainder, approximately 56/14/30 percent by groups. The training partition alone updates weights; validation selects the threshold after fixed-epoch training.

Methods: Fine-tune DNABERT; compare Random Forest, SVM, Logistic Regression and Gradient Boosting using training-only five-fold CV; choose the classical winner by mean macro F1. Optional engine/CLI capabilities include SMOTE, feature selection, tuning and soft voting; comparison mode can include installed XGBoost/LightGBM.

Data structures: Model weights, DataLoaders, training history, partition indices, fitted scikit-learn pipelines, CV scores and selected threshold.

6. Inference and prediction module

The DNABERT wrapper validates checkpoint compatibility and rejects inference when its classification head is newly initialized. It computes two logits and softmax probabilities, then applies the saved validation-selected threshold where available. The classical web route requires a pipeline trained in the current Flask process. Uploaded FASTA records must contain only A/C/G/T, identify a supported trained gene and remain within +/-5% of that reference length. Requests are limited to 16 MB and 1-100 records. Existing fitted references and transformations are reused without training on uploaded data.

Methods: Load trained state, validate inputs, transform sequences, compute class probabilities and apply the workflow's prediction rule.

Data structures: PredictionResult objects or per-record JSON with gene, header, sequence length, class and probability.

7. Performance evaluation and experiment recording module

DNABERT reports accuracy, precision, recall, binary F1, ROC-AUC, average precision and a [0, 1]-ordered confusion matrix when labels are available. Its experiment.json records configuration, package versions, source hashes, split indices/class counts, training history, threshold and held-out metrics. Classical evaluation reports macro/weighted F1, class-specific precision/recall, ROC-AUC, average precision and per-gene results after CV-based selection. Average precision is not trapezoidal PR-AUC, and the two F1 definitions should not be silently equated.

Methods: Evaluate predictions at previously fixed model settings, summarize class behavior, preserve experiment metadata and generate confusion-matrix/ROC/precision-recall visualizations.

Data structures: Metric dictionaries, confusion matrices, probability arrays, curves, comparison summaries and experiment JSON.

8. Web interface and orchestration module

app.py serves the dashboard through GET /. POST /audit returns an exact-sequence duplicate/conflict audit. POST /train delegates experiment execution to run_experiment and stores the selected fitted pipeline in the process-local _state dictionary. POST /predict validates FASTA uploads and returns predictions from that pipeline. templates/index.html visualizes dataset statistics, comparisons, plots and predictions with Bootstrap and Chart.js. Training is synchronous, a new training request replaces the active model, and a process restart clears it.

Methods: Render the page, handle JSON/multipart requests, coordinate dataset/model helpers and return validation errors or result dictionaries.

Data structures: HTTP settings objects, uploaded FASTA streams, process-local fitted pipeline, dashboard JSON and chart data.

9. Configuration, quality checks and system boundaries

utils.py provides graph-pipeline configuration, validation helpers and logging. Seeds support repeatable path sampling and partition selection; regression suites exercise checkpoint safeguards, partition behavior, preprocessing boundaries, audit output and upload handling. The current graph conversion simplifies SCC ordering, orientation and overlaps, while prefix truncation can omit an allele of interest. The classical audit exposes exact duplicate overlap but does not automatically block or group those sequences. Neither workflow establishes biological-family independence, calibrated disease risk or clinical validity.

Methods: Centralize settings, emit actionable failures, save reproducibility evidence and test software behavior. Verify annotations, biologically appropriate windows and independent cohorts before interpreting scientific results.

Data structures: Configuration objects, log messages, assertions/tests, model artifacts and documented dataset/experiment provenance.

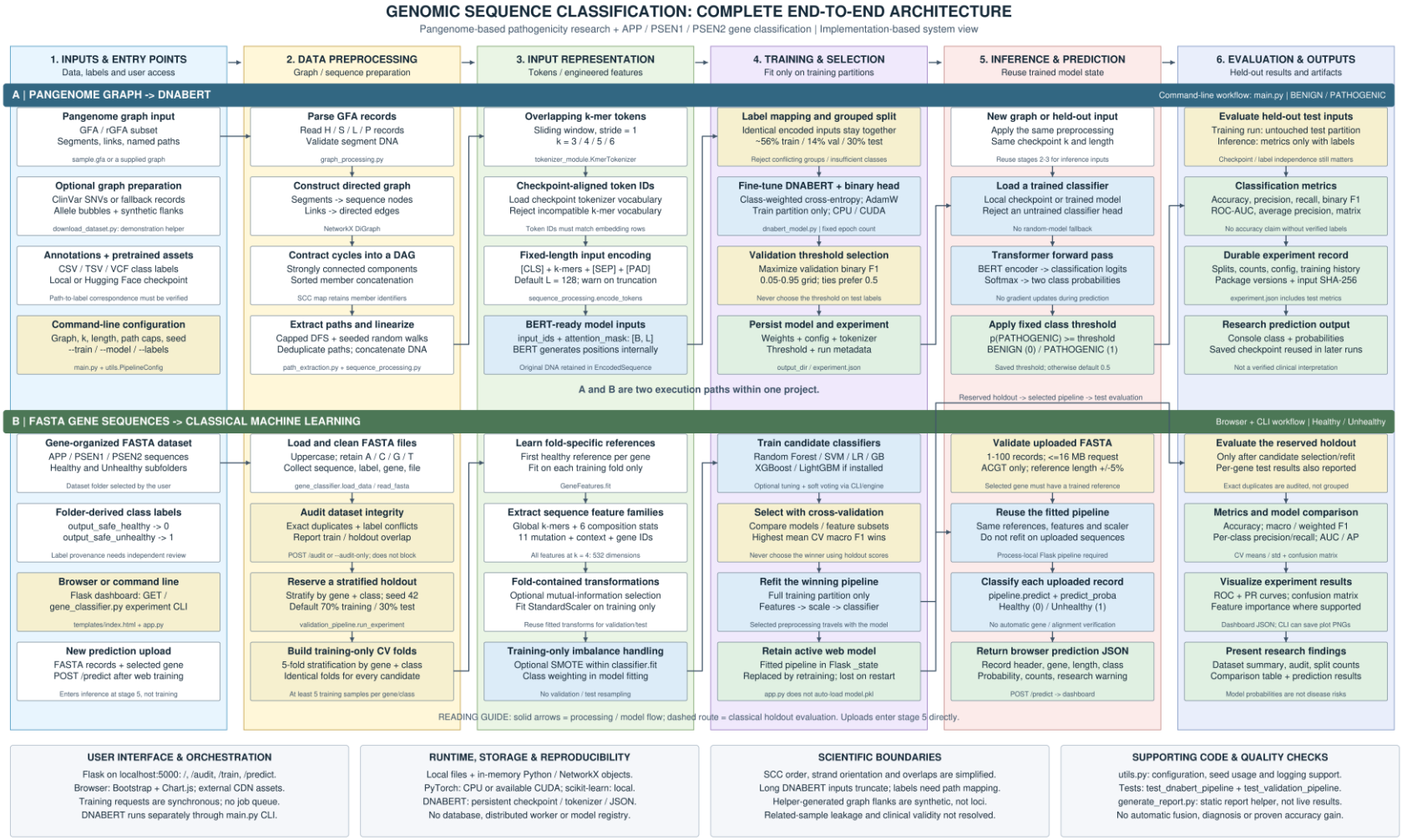


Figure 2.1: High-Level Block Diagram of the Transformer-Driven Pangenomic Framework

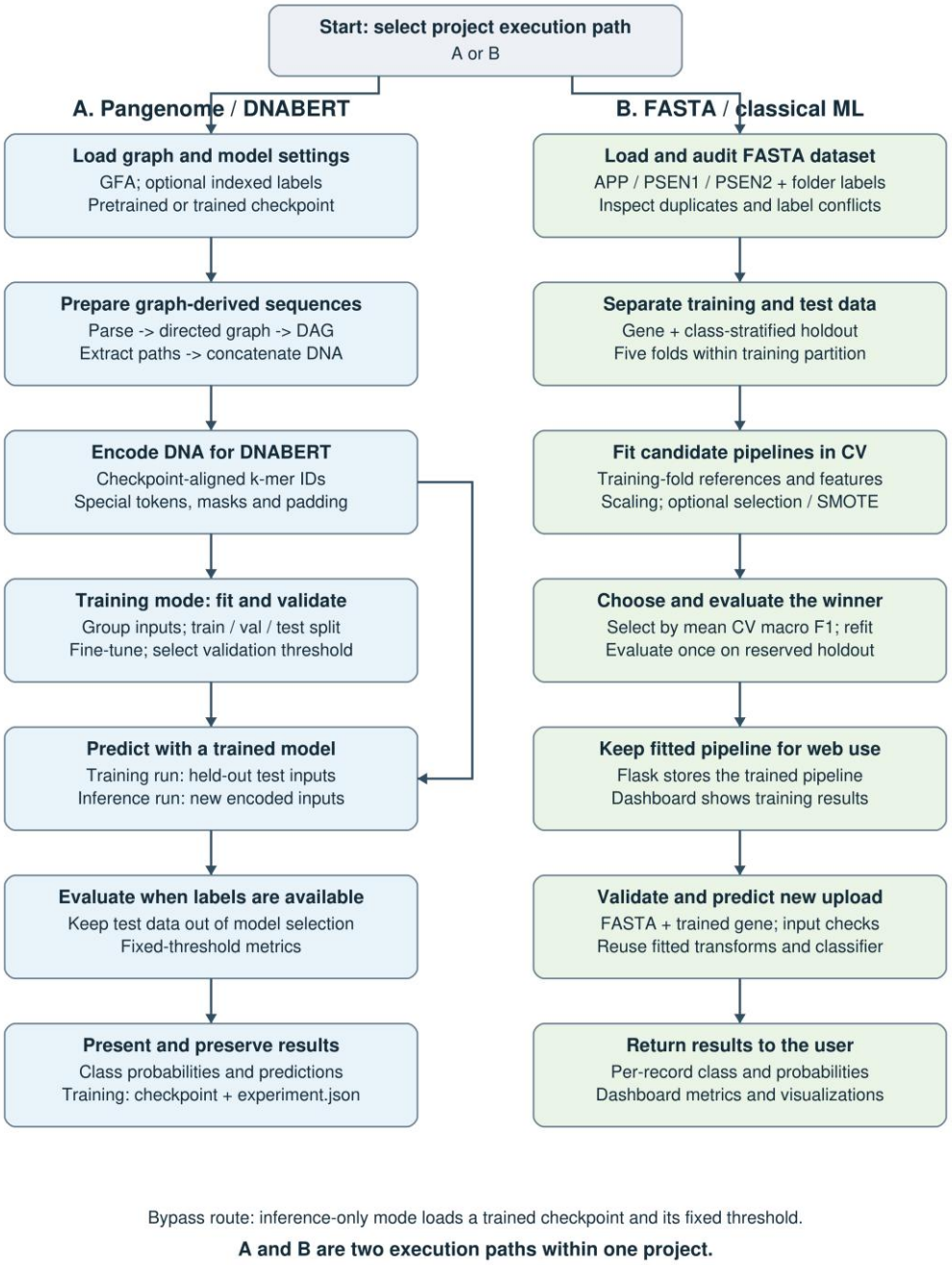


Figure 2.2: System Flowchart of the Genomic Variant and Gene Sequence Classification Process

Chapter 3

Datasets

APP, PSEN1 and PSEN2 FASTA collection

The classical workflow uses local FASTA files under Dataset\Dataset. Each supported gene has output_safe_healthy and output_safe_unhealthy subfolders. An inventory on 19 October 2026 found 46 APP files (23 per class), 156 PSEN1 files (78 per class) and 18 PSEN2 files (9 per class), for 220 files in total: 110 per class. These are file-system counts, not verified counts of independent patients or unique biological variants. Files omitted by the loader, duplicate sequences and related/generated samples can affect the effective dataset size.

Graph and annotation inputs

The DNABERT workflow accepts sample.gfa, clinvar_pangenome.gfa or a compatible supplied graph. CSV/TSV labels are linked through extracted sequence indices, and VCF annotations are indexed in record order. There is no automatic coordinate-based mapping between VCF variants and generated graph paths. The graph parser does not enforce APP/PSEN1/PSEN2 membership, so an Alzheimer's-focused experiment requires separately verified gene-specific graph inputs and annotations.

download_dataset.py is a demonstration helper, not a reference-genome reconstruction tool. It creates reference/alternate allele bubbles using variant descriptors with seeded random flanking/context DNA; it can fall back to built-in descriptors. Such generated contexts must not be described as authentic genomic loci. A filename containing ClinVar does not itself establish correct annotation provenance, valid label assignment or meaningful biological sequence context.

Preparation and partition policy

For the classical path, the loader uppercases sequence text, retains A/C/G/T characters and concatenates sequence lines. The default test fraction is 30%, stratified by gene and class; five-fold validation operates only inside the remaining training partition. Exact duplicates, label conflicts and train/test overlap are reported by the audit, but source/family grouping is not enforced. For DNABERT, identical encoded inputs are grouped, conflicting group labels stop training, and every train/validation/test partition must contain both classes.

Input and target representation

Graph-derived DNA becomes checkpoint-aligned k-mer IDs and attention masks with targets BENIGN (0) or PATHOGENIC (1). Classical DNA becomes engineered feature vectors with targets Healthy (0) or Unhealthy (1). Neither label source automatically validates the other. Prediction uploads must identify a supported gene and pass sequence-content/length checks. No external cohort, clinical label verification or held-out biological performance was established while preparing this design document.

Chapter 4

Technology Stack

The implementation combines graph processing, DNA sequence encoding, transformer fine-tuning, classical machine learning and browser-based experimentation. The stack below distinguishes core dependencies from optional experimental and reporting components.

Category	Technology / Component
Programming language	Python for the implemented pipeline, model wrappers, dataset helpers and Flask application.
Graph processing	NetworkX for directed graphs, SCC contraction, DAG checks and path traversal.
Deep learning	PyTorch and Hugging Face Transformers; compatible DNABERT / BertForSequenceClassification checkpoint.
Sequence representation	Checkpoint-aligned overlapping k-mers, special tokens, fixed-length token IDs and attention masks.
Classical ML	scikit-learn pipelines: Random Forest, SVM, Logistic Regression and Gradient Boosting; training-fold feature fitting and scaling.
Optional experiments	imbalanced-learn for SMOTE; installed XGBoost/LightGBM; CLI/engine tuning, feature selection and soft voting.
Biological input	GFA/rGFA subset, CSV/TSV/VCF annotation files, local APP/PSEN1/PSEN2 FASTA collection; Biopython for upload parsing.
Numerical processing	NumPy arrays and feature matrices; PyTorch integer tensors and probability outputs.
Web application	Flask routes with HTML, JavaScript, Bootstrap, Font Awesome and Chart.js dashboard assets.
Evaluation	scikit-learn metrics, five-fold stratification, held-out testing, per-gene summaries, ROC/PR curves and confusion matrices.
Plots and reporting	Optional matplotlib/seaborn for CLI plot files; generate_report.py uses fpdf2 for a static report helper.
Runtime and persistence	Local CPU or available CUDA for DNABERT; file-based model/tokenizer/JSON artifacts; active web pipeline in process memory.
Reproducibility / testing	Seeded experiments, input SHA-256 hashes, package versions, validation guards and unittest regression suites.

Table 4.1: Technology Stack for the Transformer-Driven Pangenomic Framework